
```
partial_words_end = [loc for loc in locs if loc[0] not in spacestarts_affine \
and loc[1] in spacestarts]
```

Now we can start inserting spaces.

Finding First and Second Halves of Potential Words

Let's start by inserting a space into `oneperfectly`. We'll define a variable called `loc` that stores the location of `oneperfectly` in our text:

```
loc = between_spaces_notvalid[0]
```

We now need to check whether any of the words in `partial_words` could be the first half of `oneperfectly`. For a valid word to be the first half of `oneperfectly`, it would have to have the same beginning location in the text, but not the same ending location, as `oneperfectly`. We'll write a list comprehension that finds the ending location of every valid word that begins at the same location as `oneperfectly`:

```
endsofbeginnings = [loc2[1] for loc2 in partial_words if loc2[0] == loc[0] \
and (loc2[1] - loc[0]) > 1]
```

We've specified `loc2[0] == loc[0]`, which says that our valid word must start at the same place as `oneperfectly`. We've also specified `(loc2[1]-loc[0])>1`, which ensures that the valid word we find is more than one character long. This is not strictly necessary, but it can help us avoid false positives. Think of words like *avoid*, *aside*, *along*, *irate*, and *iconic*, in which the first letter could be considered a word on its own but probably shouldn't be.

Our list `endsofbeginnings` should include the ending location of every valid word that begins at the same place as `oneperfectly`. Let's use a list comprehension to create a similar variable, called `beginningsofends`, that will find the beginning location of every valid word that ends at the same place as `oneperfectly`:

```
beginningsofends = [loc2[0] for loc2 in partial_words_end if loc2[1] == loc[1]
and \ (loc2[1] - loc[0]) > 1]
```

We've specified `loc2[1] == loc[1]`, which says that our valid word must end at the same place as `oneperfectly`. We've also specified `(loc2[1]-loc[0])>1`,